Битови манипулации и хардуерни оптимизации за бързи трансцендентни функции в Edge и IoT устройства

Г. Петров

Bit Manipulations and Hardware Optimizations for Fast Transcendental Functions in Edge and IoT Devices

G. Petrov

Department of Telecommunications, New Bulgarian University, 21 Montevideo St., 1618 Sofia, Bulgaria, gpetrov@nbu.bg

Key Words: *Bit manipulations; hardware optimizations; Edge devices; lookup tables (LUT); CORDIC; fast inverse square root; embedded machine learning.*

Abstract. *Fast and energy-efficient evaluation of transcendental functions (for example, logarithm, exponential, square root, and normalization) is critical for real-time signal processing and on-device AI inference in Edge and IoT systems. This paper surveys and experimentally validates a set of lightweight techniques – primarily IEEE-754 based bit-level manipulations (bit tricks) – alongside code- and hardware-level optimizations (loop unrolling, DMA usage, and other low-level optimizations) that dramatically accelerate these functions on resource-constrained microcontrollers. We describe simple but effective bit hacks for fast approximations of* $\log_2(x)$, 2^x, $\sqrt{x}$ *and* $1/\sqrt{x}$ *(including the well-known "Quake" inverse square root), explain their theoretical basis in floating-point representation, and show how modest corrections to the mantissa (linear and low-order polynomial) can substantially improve accuracy while retaining most performance gains. Benchmarking across representative platforms (AVR, Cortex-M variants with/without FPU, ESP32, and RISC-V boards) demonstrates up to 10× speedups relative to standard library implementations, with typical accuracy in the range of 5-6 bits for raw bit-trick estimates and much improved error after correction terms. We discuss practical integration patterns, trade-offs between latency, memory footprint, and numerical fidelity, and recommend when to combine bit tricks with LUTs or CORDIC for broader function coverage. Finally, we outline use cases (embedded ML feature scaling, entropy computation, normalization in DSP and robotics) and caveats (unsuitability for high-precision scientific/financial calculations and for periodic functions like sin/cos), providing guidelines for safe deployment in production Edge/IoT projects.*

1. Въведение

С нарастващото приложение на Edge и IoT устройства в индустрията, дома и медицината възниква нужда от локална обработка и мрежова свързаност [1] на данни при минимални ресурси и енергопотребление. Много алгоритми [2] използват трансцендентни функции (log, exp, sqrt), които са бавни без FPU (Floating Point Unit). Сред ефективните подходи се открояват LUT (Look up Tables), CORDIC (Coordinate Rotation Digital Computer) и особено битовите манипулации (bit trick) техники, базирани на формата IEEE754, които осигуряват бързи оценки с минимална изчислителна тежест. В статията са анализирани тези подходи, сравнени са чрез реални експерименти и са предложени практични насоки за прилагането им върху платформи като AVR, Cortex-M и ESP32. Дискутираните в статията техники не са част от стандартната Arduino библиотека и биха представлявали интерес за нови разработчици на IoT с ограничени ресурси и енергиен бюджет.

1.1. Мотивация и актуалност

През последното десетилетие Edge и IoT устройствата бележат бърз растеж, изпълнявайки локални изчисления, мониторинг и ML (многоядрени контролери [3]) задачи в реално време при ограничени ресурси. За разлика от класическите платформи с FPU и оптимизирани библиотеки много вградени системи разчитат на целочислена аритметика. Стандартни методи като LUT и CORDIC често не са достатъчни при висока изчислителна плътност, каквато е обработката на изображения от мобилни роботи [4]. В този контекст битовите манипулации (bit trick) предлагат изключително бързо и компактно решение за функции като $\log_2(x)$, exp(x), sqrt(x), особено при наличен FPU.

1.2. Микроконтролери със и без FPU (ARM, ESP32, STM32, AVR)

В основата на повечето Edge и IoT устройства стоят микроконтролери и процесори с различна архитектура и ниво на изчислителна мощност. Плаващата запетая е стандартизирана през 1985 г. чрез IEEE 754, а става широко достъпна във вградените платформи след 2000 г.

(Cortex-M4F, ESP32 и т.н.) Във масовите IoT устройства до днес целочислената аритметика продължава да бъде доминираща, особено при отсъствие на FPU [5]. Основните хардуерни ограничения, с които разработчиците се сблъскват, включват (*табл. 1*):

- **Ограничена RAM и Flash памет.** Типичните устройства разполагат с 2-512 KB RAM и до няколко MB Flash, което е проблем за големи LUT.
- **Липса на хардуерен FPU.** Много 8- и 32-битови микроконтролери (например AVR, ARM Cortex-M0/M3, някои RISC-V и ESP32-C3) нямат хардуерна поддръжка за плаваща запетая, което прави изчисленията бавни.
- **Ограничена тактова честота.** Повечето устройства работят на 8-240 MHz.
- **Енергийна ефективност.** Edge устройствата често работят на батерии или в енергийно критични среди, където всяка изчислителна оптимизация е от значение.

Табл. 1. Сравнение на хардуерни архитектури и възможности за прилаганена различни оптимизации

Платка/ MCU	Архитектура	FPU	Типични оптимизации
Arduino Uno	AVR 8-bit	Не	LUT, CORDIC, fixed-point
Arduino Nano 33 IoT	ARM Cortex-M0+	Не	LUT, fixed-point
STM32F103	ARM Cortex-M3	Не	CORDIC, LUT
STM32F411, F746	ARM Cortex-M4F/M7	Да	Bit trick, float, LUT
ESP32	Xtensa LX6	Да	Bit trick, float, DMA
ESP32-C3	RISC-V	Не	LUT, fixed-point
Longan Nano	RISC-V	Не	LUT, CORDIC, integer

1.3. Специфика на Edge и IoT приложения

Edge и IoT устройствата често изпълняват задачи като:

- Локална обработка на сензорни данни (филтриране, нормализация)
- Генериране на сигнали (PWM, синус, шум)
- Класификация и обработка на изображения (ML)
- Калкулиране на ентропия и други статистики в реално време

Във всички тези случаи се изискват трансцендентни функции, които са:

- Бързи (реално време)
- С малък разход на енергия
- С минимална памет
- Често без достъп до FPU

2. Битови манипулации за трансцендентни функции

2.1. Теоретични основи: IEEE754 и bit trick техники

Битовите манипулации (bit trick техники) използват особеностите на формата за представяне на числа с плаваща запетая по стандарта IEEE754, които са ефективни, нискоразходни методи за приближаване на логаритми, експоненти и квадратни корени. Директната манипулация на експонентата и мантисата чрез побитови преобразувания позволява реализация на редица функции с постоянна времева сложност и без използване на тежки математически библиотеки [6]. В 32-битов float:

- 1 бит е за знак;
- 8 бита за експонента;
- 23 бита за мантисата.

Тъй като експонентата е свързана директно с логаритмичното мащабиране на числото, чрез битова манипулация можем много бързо да получим приблизителни стойности на $\log_2(x)$, 2^x, sqrt(x) и други функции – без нужда от тежки аритметични операции или цикли.

Пример 1. Бързо изчисление на $\log_2(x)$

```
float fast_log2(float x) {
    union { float f; uint32_t i; } vx = { x };
    return (float)((vx.i >> 23) & 255) - 127; }
```

- Тук просто се извлича експонентата от битовете на float числото.
- Точност: ~5-6 бита, мигновено време за изпълнение.

Изчислението на експоненциалната функция exp(x) във вградени системи обикновено изисква или хардуерен FPU, или достъп до специализирани математически библиотеки. Когато това не е възможно – например при платформи с ограничени ресурси или в ситуации, изискващи екстремно бързо изчисление, може да се приложи битово приближение, базирано на свойството (1):

(1) $\exp(x)=2^{x*\log_2(e)} \approx \text{pow2}(x*1.442695)$.

Това позволява използване на вече позната bit trick техника за пресмятане на 2^x, от горния пример, адаптирана чрез умножение с константата $\log_2(e)\approx1.442695$.

```
float fast_exp(float x) {
    return fast_pow2(x * 1.442695f); // log2(e) ≈ 1.442695 }
```

Пример 2. Бързо изчисление на 2x

```
float fast_pow2(float x) {
    union { uint32_t i; float f; } vx;
    vx.i = (uint32_t)((x + 127.0f) * (1 << 23));
    return vx.f; }
```

- Пресмята float чрез обръщане на $\log_2(x)$ bit trick.
- Ограничена точност, но изключително бързо.

Пример 3. Бързо изчисление на sqrt(x), 1/sqrt(x) (Quake Trick)

```
float fast_inv_sqrt(float x) {
    float xhalf = 0.5f * x;
    int i = *(int*)&x;  // Битов хакинг
    i = 0x5f3759df - (i >> 1); // Магическа константа
    x = *(float*)&i;
    x = x*(1.5f - xhalf*x*x); // Newton итерация
    return x; }
```

Използва се в графики, игри и ML за бързо нормализиране на вектори.

2.2. Алгоритми: fast $\log_2(x)$, 2^x, sqrt(x), abs(x), sign(x)

- **abs(x):** Побитово се нулира знаковият бит (най-старшият).
- **sign(x):** Може да се вземе чрез маска на най-старшия бит или чрез сравнение с 0.
- **int $\log_2(x)$:** Чрез бързото търсене на най-старшия бит (bit scan, builtin_clz()).

Забележка. Тези техники работят **само при наличен FPU и IEEE754 формат**, основно с float32.

2.3. Ограничения и приложимост

- **Ограничена точност:** 5-6 бита (не за научни изчисления).
- **Неуниверсалност:** Неприложими за синус, косинус и други периодични функции.
- **Нужда от FPU:** Не могат да се използват при чисто целочислени архитектури.
- **Лесна интеграция:** Подходящи за embedded ML, entropy, scaling, DSP.
- **Енергийно критични:** edge устройства с float32 поддръжка (ARM M4F, ESP32, STM32F4/7).

Табл. 2. Функции, за които съществуват bit trick или подобни оптимизации

Функция	Метод/ Приближение	Приложение
$\log_2(x)$	bit trick по експонентата	ML, ентропия, графика
2^x	обратен bit trick	Sigmoid, exp scaling
sqrt(x)	побитова апроксимация	DSP, ML, норма, енергия
1/sqrt(x)	Fast inverse sqrt (Quake)	Векторна нормализация, physics, vision
exp(x)	косвен	чрез $2^{x \cdot \log_2}(e)$ работи с bit trick за pow2()
abs(x)	bit mask (нулване на знака)	Безусловна логика, сигнали
sign(x)	побитово извличане на знака	Класификация, управление, плавност в игрови контролери
clamp(x)	безусловно ограничаване	Активни защити, обхват
int $\log_2(x)$	bit scan (MSB index)	Ентропия, бинове, иерархии
round(x)	add + bias & truncate	DSP, фиксирано време

С разпространението на 16-битовата плаваща запетая (float16) в ML, графика и Edge inference, битовите оптимизации стават актуални и за този формат. Float16 е по-компактен вариант на IEEE754, използван основно в GPU и AI хардуер с достъп до битовете чрез CUDA или OpenCL. Bit trick техниките са ефективни за бързи логаритми и степени при наличен FPU, но не са приложими за синус и косинус и не осигуряват достатъчна точност за научни и финансови задачи.

3. Експерименти: Bit Trick в Arduino

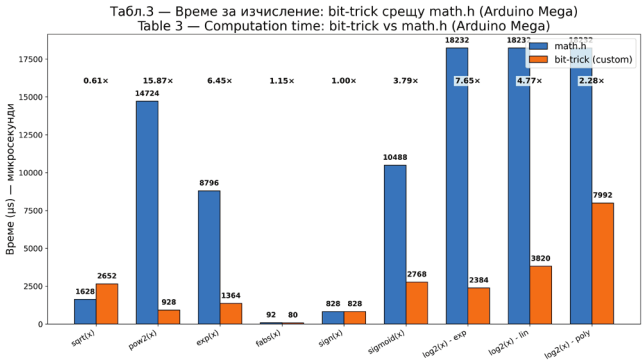
3.1. Методология

За да бъде оценена практическата ефективност на битовите техники за пресмятане на логаритъм по основа две, е проведен експеримент върху платформа Arduino Mega 2560 (ATmega2560, 16 MHz, 8 KB RAM). Сравнени са три bit trick реализации на $\log_2(x)$ – *табл. 3* и *фиг. 1:*

- Класически вариант (експонента само).
- С линейна поправка на мантисата.
- С полиномиална поправка на мантисата.

Табл. 3. Време за изчисление и грешка – Bit Trick срещу math.hq Arduino Mega

Функция	math.h време (us)	bit trick време (us)	Средна грешка	Макс. грешка
sqrt(x)	1628	2652	0.0020	0.0053
pow2(x)	14724	928	10.87	57.28
exp(x)	8796	1364	8.54	48.42
fabs(x)	92	80	0.0000	0.0000
sign(x)	828	828	0.0000	0.0000
sigmoid(x)	10488	2768	0.050	0.082
$\log_2(x)$ – exp	18232	2384	0.456	0.982
$\log_2(x)$ – lin	18232	3820	0.054	0.086
$\log_2(x)$ – poly	18232	7992	0.038	0.186



Фиг. 1. Време за изчисление и грешка – Bit Trick срещу math.hq Arduino Mega

Забележка 1. Quake trick (fast_inv_sqrt) реално съдържа: манипулация на битове (memcpy/cast), аритметични операции, **умножение** и **изваждане**, Newton-Raphson итерация. Quake Trick е оптимален за платформи с бързи битови операции и FPU: ARM Cortex-M4F, ESP32, STM32.

Забележка 2. За функцията $\log_2(x)$ са реализирани три варианта на bit trick апроксимация: с използване само на експонентата, с линейна поправка на мантисата и с полиномиална поправка (Schraudolph, 1999). Bit trick е над седем пъти по-бърз от math.h, но средната грешка (~0.46) е висока за точни изчисления. Линейната и полиномиалната поправка намаляват драстично грешката (~0.05 и ~0.04 съответно), като запазват значителното ускорение.

$\log_2(x)$ – exp: само експонентата

$\log_2(x)$ – lin: експонента + линейна поправка за мантисата

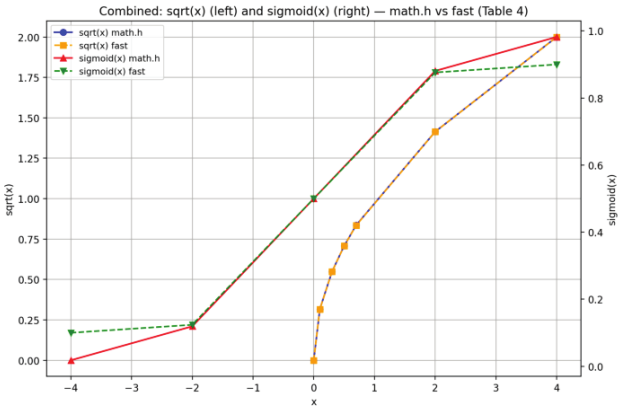
$\log_2(x)$ – poly: експонента + полиномиална поправка

Табл. 4. Примерни изходни стойности (sqrt, pow2, sigmoid) и стандартни функции

x	sqrt(x) math.h	sqrt(x) fast	pow2(x) math.h	pow2(x) fast	sigmoid(x) math.h	sigmoid(x) fast
-4.0	–	–	–	–	0.0180	0.1000
-2.0	–	–	0.2500	0.0000	0.1192	0.1238
-1.6	–	–	0.3121	0.0000	–	–
0.0	0.0000	0.0000	1.0000	1.0000	0.5000	0.5000
0.1	0.3162	0.3157	–	–	–	–
0.3	0.5477	0.5477	–	–	–	–
0.5	0.7071	0.7069	–	–	–	–
0.7	0.8367	0.8354	–	–	–	–
2.0	1.4142	1.4139	4.0000	4.0000	0.8808	0.8762
4.0	2.0000	2.0000	16.0000	16.0000	0.9820	0.9000

3.2. Анализ и дискусия

Резултатите показват, че дори най-елементарният Bit Trick (само експонента) [7] ускорява изчисленията 7-8 пъти спрямо math.h, макар и с по-голяма грешка – *табл. 4* и *фиг. 2*. Линейната и полиноминалната поправка на мантисата подобряват значително точността (до 0.038). Bit trick вариантите са 4-15 пъти по-бързи при често използвани функции като pow^2, exp, sigmoid и sqrt, а при sqrt(x) точността е практически идентична с math.h. При $pow^2(x)$ и exp(x) точността спада при отрицателни аргументи, но за $x \geq 0$ резултатите са надеждни. За sigmoid(x) максималната грешка е около 0.08, което е напълно приемливо за ML и edge inference. При fabs(x) и sign(x) няма разлика между стандартната и битовата реализация. В заключение Bit Trick оптимизациите са отличен избор за ускорено изчисление на специални функции в Edge и IoT системи, когато се търси баланс между производителност и приемлива точност, дори и ML приложения [8].



Фиг. 2. Сравнение на скорост при изчисление на sqrt – ляво, и sigmoid – дясно

4. Хардуерни и софтуерни оптимизации

В Edge и IoT устройствата с ограничени ресурси ефективността на изчисленията зависи както от математическия подход, така и от хардуерните и кодовите оптимизации. Битови манипулации, циклово разгръщане, DMA и побитови заместители на аритметични операции ускоряват допълнително изчисленията и пестят енергия. Съчетавайки тези техники, могат да се реализират критични real-time алгоритми дори върху най-ограничения хардуер, като се запазва нужната точност. В тази секция се представят основните практични хардуерни и софтуерни оптимизации за Edge и IoT платформи.

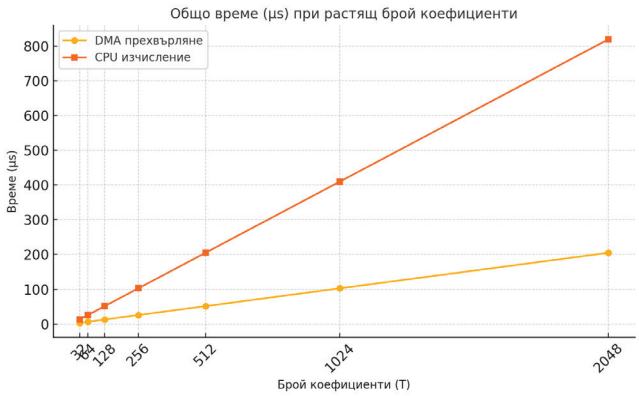
4.1. Циклово разгръщане (Loop Unrolling)

Loop unrolling е техника, при която итерациите на цикъл се „разгъват“ в последователни инструкции. Това намалява броя на условните проверки (cmp, jump), подобрява ефективността на процесорния pipeline и води до по-последователен достъп до паметта (кеш при CPU). Методът е особено удобен при реализация на FIR (Finite Impulse Response) филтри, защото всички коефициенти са фиксирани, както и други структури с фиксирана дължина и известни коефициенти. Пример за FIR филтър n размерно ядро:

(2) $y[n]=b_0 \cdot x[0]+b_1 \cdot x[1]+b_2 \cdot x[2]+...+b_{(n-1)} \cdot x[n-1]+b_n \cdot x[n].b_n \cdot x[n].$

FIR с цикъл	FIR без вътрешен цикъл

float b[T] = {0.1, 0.2, 0.3, 0.2, 0.1};
float x[N]; // входен N+T-1 елемента
float y[N];
for (int n = 0; n < N; n++) {
y[n] = 0;
for (int k = 0; k < T; k++) {
y[n] += b[k] * x[n - k]; }}
	float b[T] = {0.1, 0.2, 0.3, 0.2, 0.1};
	float x[N]; // вход N+T-1 елемента
	float y[N];
	for (int n = 4; n < N; n++) {
	y[n] = 0.1 * x[n] + 0.2 * x[n - 1] + 0.3 * x[n - 2] + 0.2 * x[n - 3] + 0.1 * x[n - 4];
	}



Фиг. 3 Сравнение на броя на спестените операции (време) при употреба на loop unrolling и същата, съчетана с DMA трансфер. Този метод води до линейно ускорение и ако е приложим, винаги може да бъде прилаган. При големи цикли следва да се внимава с елементите, които разгръщаме поради опасност от загуба на памет (loops > 32)

На *фиг. 3* е показано как нараства броят на спестените условни преходи при **loop unrolling**, когато **броят на филтърните коефициенти (T)** се увеличава, а **входният сигнал е с фиксирана дължина (N = 100)**. Чрез комбинация с DMA тази скорост може да бъде намалена драстично, процесорът извършва единствено аритметични операции и не мести променливите.Цикловото разгръщане при по-сложен код обикновено се осъществява чрез макрос и на разгръщаната функция.

4.2. Побитови заместители на аритметични операции

Умножението по степен на две 2^n може да бъде заменено с побитово изместване, което е по-бързо и хардуерно по-ефективно, подобна техника беше дискутирана при реализацията на CORDIC алгоритъма: **x = x << n вместо x = x * (2^n).** Деление на 2 се замества с **побитово отместване** надясно. Тази операция се използва често при линеен CORDIC режим за реализация на приближени деления и умножения: **x = x >> 1 вместо x/2**. Нулирането на стойност (масиви) може да се извърши чрез побитова операция **XOR**, а не чрез условна логика: **x = x xor x вместо x = 0**. Маскиране на знака: **x & 0x7FFFFFFF** за взимане на абсолютна стойност при int32_t. При наличие на достатъчно памет размерите на елементите в масива следва да се подравняват по размера на шината за данни, 16, 32, 64, това е важно условие за изпълнение в реално време при игри.

4.3. Интеграция с други техники (CORDIC, LUT)

Често битовите и структурните оптимизации се комбинират с други числени подходи:

- **Bit trick** – за log2, pow2, sqrt, ML scaling
- **LUT** – за log, exp, sigmoid, entropy (където точността е критична)
- **CORDIC** – за синус, косинус, арктангенс, когато няма FPU

Пример за това е реализацията на IIR (Infinite Impulse Response) филтър с целочислена аритметика и побитови отмествания, което го прави особено подходящ за нискоенергийни embedded системи. Коефициентите са мащабирани, за да се избегне плаваща запетая [9].

(3) $y[n]=a_0 \cdot x[n]+a_1 \cdot x[n-1]+a_2 \cdot x[n-2]-b_1 \cdot y[n-1]-b_2 \cdot y[n-2]/(4096,)$

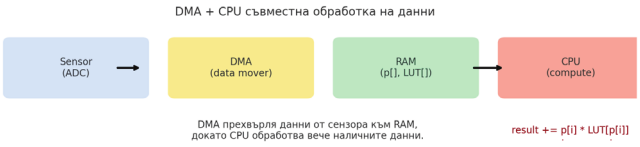
където:

$a_0 = 4096 / 4096 = 1.0$
$a_1 = –6211 / 4096 \approx –1.516$
$a_2 = 3598 / 4096 \approx 0.879$
$b_1 = –6627 / 4096 \approx –1.616$
$b_2 = 4096 / 4096 = 1.0$

Всички коефициенти са мащабирани във фиксирана точка (Q12), а делението се извършва с побитово отместване (>>12), което елиминира бавните операции с плаваща запетая.

4.4. DMA асистирани изчисления

DMA (Direct Memory Access) – *фиг. 4*, може да се използва не само за трансфер на данни, но и за асистиращ механизъм в изчислителни потоци, например за изчисление на CRC (Cycle Redundancy Check) контролни суми [10]. При подходяща организация на паметта и таблиците (LUT) DMA може да захранва CPU с нужните стойности за обработка (например p[i] и log(p[i])), като процесорът извършва само последващи аритметични операции като * и +. Само чрез употребата на DMA е възможно постигане на максимални скорости на трансфер на данни от АЦП към USB/Ethernet интерфейси, каквито например са нужни за реализация на спектрални анализатори, бързи дата логери и т.н.



Фиг. 4. Използване на DMA за улесняване на работата на основния изчислителен софтуер

5. Критична дискусия

Въпреки представените предимства всеки от използваните методи (LUT, bit-trick) има своите ограничения, които трябва да се вземат предвид в реални системи, особено нискоенергийни IoT [11]:

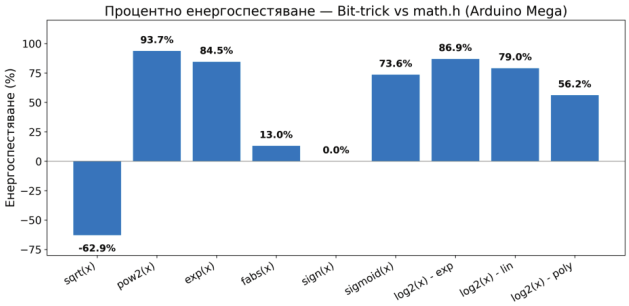
- **Ограничена точност.** Приблизително 5-6 бита при log2(x), pow2(x), което ги прави неподходящи за критични и научни изчисления.
- **За по-висока точност и скорост изискват FPU и float32 поддръжка.** Неподходящи за fixed-point (AVR, някои ARM M0/M3, RISC-V без FPU).
- **Неуниверсалност.** Неподходящи за периодични функции (sin, cos), защото IEEE 754 представянето не позволява побитово манипулиране на периодични стойности.
- **При различни техники за оптимизация.** Необходима е експериментална селекция на алгоритмичните ускорения и целенасочено изключване/контрол на етапа на компилация и хардуерните оптимизации, за да се гарантира възпроизводимостта и коректността на числените резултати.

6. Енергопотребление

За да придобием представа за допустимото редуциране в енергоспестяването, което се постига чрез ползването на приближените техники за пресмятане в Edge и IoT устройствата, ще използваме обобщен израз (4). Като разглеждаме общата консумирана енергия като функция на времето за изпълнение на една или друга операция чрез math.h, CORDIC, Bit-trick и другите разгледани операции.

(4) $\text{Savings\%} = (1 - \frac{E_{total_{optimized}}}{E_{total_{math.h}}})$.

Битовите манипулации осигуряват между 40% до 95% енергоспестяване, като това може да е отрицателно за някои функции като sqrt. Добавянето на линейна корекция води до 50%-85%, а полиномиалната редуцира това до ~30%-70%. От друга страна, при наличие на достатъчно свободна памет употребата на таблици за заместване води до ~50%-99% подобрена енергоефективност, като в случаите на специфични задачи често е по-добре употреба на външен цифров генератор. CORDIC се използва при системи където е необходима повишена точност и изчисление на периодични функции като sin(x) ~20%-70%. Тези техники при наличие на памет и ресурси, съчетани с кодови оптимизации (inlining, unrolling, глобални променливи и други), могат да донесат допълнителни ~5% до 40% подобрено енергопотребление, а при наличие на хардуерен FPU и специални инструкции това води до още 10% до 90% подобрена енергоконсумация. Примерни данни за употреба на bit-trick, касаещо редукцията на енергопотреблението, са дадени на *фиг. 5*.



Фиг. 5. Процентно енергоспестяване при изпълнение чрез използване на bit-trick операции

7. Заключение

Бързото и енергийно ефективно изчисление на функции като логаритъм, експонента и корен е ключово за Edge и IoT устройства с ограничени ресурси. В статията се разглеждат битови манипулации (bit trick) и хардуерни оптимизации (например DMA, циклово разгръщане), които позволяват до 10× ускорение спрямо стандартни библиотеки при точност от 5-6 бита. Представени са алгоритми, базирани на IEEE754 формат и тяхната приложимост в реални задачи като нормализация, ентропия и вградени AI. Разгледани са ограниченията и са предложени насоки за ефективна интеграция в Edge/IoT системи.

В статията са анализирани енергийно ефективни методи за бързи трансцендентни изчисления в среди с

ограничени хардуерни и енергийни ресурси, типични за Edge и IoT устройства. Основният акцент е върху битови техники (bit trick) и хардуерни оптимизации като цикло-во разгръщане и побитови операции, които позволяват значително ускорение при изчисления с плаваща запетая и са част от алгоритми като CORDIC, IIR, FIR филтри.

Резултатите от тестове с различни архитектури (AVR, ARM, ESP32, RISC-V) показват, че правилната комбинация от тези техники може да гарантира оптимален баланс между скорост, точност и енергийна ефективност. Представените решения са приложими за реални изчислително интензивни AI задачи в Edge и IoT [12] проекти, където се търси максимална изчислителна ефективност при ограничени хардуерни ресурси.

Литература

1. Toskov, B., A. Toskova. Multi-agent Sensor Network. – *Automatica and Informatics*, LII, 2019, No. 2, 23-26, ISSN 0861-7562 (Print), ISSN 2683-1279 (Online). Тосков, Б., А. Тоскова. Мултиагентна сензорна мрежа. – *Автоматика и информатика*, LII, 2019, № 2, 23-26, ISSN ISSN 0861-7562 (Print), ISSN 2683-1279 (Online).

2. Slavova, V. & S. Ivanov. Introduction to Algorithms and Programming. Sofia, New Bulgarian University Publishing House, 2004, ISBN 954-535-297-3. Славова, В. & С. Иванов. Увод в алгоритмите и програмирането. София, Издателство Нов български университет, 2004. ISBN 954-535-297-3.

3. Pavlitov, C., L. Spirov, Y. Gorbounov, N. Stefanov. Parallel Processing in Electrical Drives Control, Computer Science. 3rd International Bulgarian-Turkish Conference, Istanbul, Turkey, 2006.

4. Slavov, D. Mobile Robot Control Using Machine Vision Smart Camera. – *Automatica and Informatics*, LII, 2019, No. 2, 9-16, ISSN 0861-7562 (Print), ISSN 2683-1279 (Online). Славов, Д. Управление на мобилен робот посредством смарт камера за машинно зрение. – *Автоматика и информатика*, LII, 2019, № 2, 9-16, ISSN ISSN 0861-7562 (Print), ISSN 2683-1279 (Online).

5. Milovanov, Nikolay. Case Study: LoRa/LoRaWAN as a Suitable LPWAN Choice for Pressure and Flow Monitoring in Bulgarian Water Utility Sector. 2024, 173-179, 10.1145/3674912.3674945.

6. Schraudolph, N. N. Fast Approximations for Logarithm, Exponential, Power and Inverse Root Functions (Technical Report IDSIA-22-98). IDSIA, 1999. Retrieved from https://hal.science/hal-01227877v1/document.

7. Blinn, J. F. Floating-Point Tricks. – *IEEE Computer Graphics and Applications*, 17, July-Aug. 1997, No. 4, 80-84. doi: 10.1109/38.595279, Key Words: Personal Communication Networks; Digital Arithmetic; Text Processing; Engineering Profession; Hardware;Programming Profession; Testing; Assembly.

8. Velkova, I. Unstructured Data Processing and Analysis Using Artificial Intelligence. . – *Automatica and Informatics*, LV, 2022, No. 4, 27-30, ISSN 0861-7562 (Print), ISSN 2683-1279 (Online). Велкова, И. Обработка и анализ на неструктурирани данни с помощта на изкуствен интелект. – *Автоматика и информатика*, LV, 2022, № 4, 27-30, ISSN ISSN 0861-7562 (Print), ISSN 2683-1279 (Online).

9. Harpe, P. ECG Front-End Design Simplified. Analog Dialogue, Analog Devices, 53-11, Nov. 2019. [Online]. Available: https://www.analog.com/en/resources/analog-dialogue/articles/ecg-front-end-design-simplified.html.

10. STMicroelectronics. STM32WB – Security: Cyclic Redundancy Check (CRC), 2020. Техническо ръководство. 5 юли 2025, достъпно на: https://www.st.com.

11. Milovanov, Nikolay. Case Study: LoRa/LoRaWAN as a Suitable LPWAN Choice for Pressure and Flow Monitoring in Bulgarian Water Utility Sector. 2024, 173-179. 10.1145/3674912.3674945.

12. Simeonova, Tsvetelina. Specifics in the Provision of Integrated Services in the Conditions of Convergence of IoT (Internet of Things) and AI (Artificial Intelligence) Technologies. Yearbook Telecommunications. 2024, 11. 11-22. 10.33919/YTelecomm.24.11.2.

За контакти:
Доц. д-р **Георги Петров**
Департамент "Телекомуникации"
Нов български университет
ул. Монтевидео 21, 1618 София
НБУ, бл. 2, ет. 6, офис 609
gpetrov@nbu.bg